

# **Antes del Lab — Tema 1: Anatomía de un Agente y Superficie de Ataque**

---

*Slides de apoyo técnico — se dan antes de correr el Lab 1.1*

## Qué vamos a necesitar entender

Los 3 labs de este tema ( `ch01-lab1/2/3-ollama` y sus pares `-gcp` ) están escritos en **Google ADK** (Agent Development Kit). Antes de leer el código hace falta tener claros 4 conceptos que todavía no cubrimos en detalle:

1. Qué es un **Agent** en ADK
2. Qué es el **Runner** — quién hace que el agente "corra"
3. Qué es la **memoria de sesión** ( `SessionService` ) — y qué NO hace
4. El ciclo **Planificador** → **Ejecutor** → **Herramientas** que van a ver en los logs

Con esto alcanza para leer cualquiera de los 3 `.py` de este tema.

## Qué es un **Agent** en ADK

Un **Agent** es una **definición declarativa**, no un programa que vos escribís paso a paso. Se arma con 3 piezas:

```
agent = Agent(  
    name="anatomy_lab",  
    model=LiteLlm(model="ollama_chat/qwen3.5:9b", ...),  
    instruction="REGLA OBLIGATORIA: ...",    # el "prompt de sistema"  
    tools=[read_file, write_file],          # funciones Python comunes  
)
```

- **instruction**: texto que le dice al modelo cómo comportarse (el equivalente a un system prompt)
- **tools**: una lista de **funciones Python planas** — en este seminario no hace falta ningún decorador tipo `@tool`. ADK expone esas funciones al modelo como herramientas invocables, usando su nombre, parámetros y docstring
- El **Agent** en sí **no ejecuta nada todavía** — solo describe qué modelo, qué instrucción y qué herramientas tiene disponibles

## Qué es el Runner

Si el **Agent** es la *definición* ("quién es, qué sabe hacer"), el **Runner** es quien lo **pone a correr** sobre una tarea concreta — como un director de orquesta que le da la entrada al agente, recibe cada paso que va dando, y se lo va mostrando turno a turno.

```
runner = Runner(agent=agent, app_name="anatomy-lab",
                session_service=session_service)

for event in runner.run(user_id="lab", session_id=session.id,
                       new_message=msg):
    # cada "event" es un paso del agente: puede traer un
    # function_call (el modelo decide llamar una tool)
    # o texto (una respuesta, parcial o final)
    ...
```

`runner.run(...)` no devuelve una respuesta de una sola vez: es un **generador** que va entregando *eventos* a medida que el agente piensa, llama herramientas y responde — por eso el patrón siempre es un `for event in runner.run(...)`, nunca `agent.run("texto")` directo.

## SessionService: la memoria de la conversación

El `SessionService` guarda el **historial** de una conversación (mensajes, qué tools se llamaron, qué devolvieron) bajo un `session_id`. En estos labs se usa `InMemorySessionService`: todo vive en RAM, se pierde al terminar el proceso.

**Punto importante, verificado en el código real de `anatomy_lab11.py`:**

```
def run_task(task: str) -> str:
    session = _session_service.create_session_sync(
        app_name="anatomy-lab", user_id="lab", session_id=str(uuid.uuid4()))
    ...
```

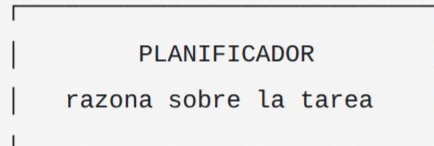
Cada llamada a `run_task()` crea un `session_id` **nuevo** (`uuid.uuid4()`). La memoria sí se acumula *dentro* de una misma llamada (por eso el agente no repite `write_file` para confirmar lo que acaba de escribir), pero **no persiste entre una llamada a `run_task()` y la siguiente** — la Tarea 2 del Lab 1.1 tiene que volver a llamar `read_file` porque no tiene memoria de la Tarea 1, solo el archivo que quedó guardado en `/tmp`.

## El ciclo que van a ver en los logs

Los 3 labs imprimen su propia evidencia de cada paso. Así se lee el trace:

Tarea del usuario

|



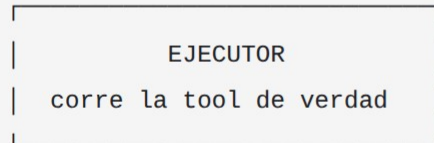
PLANIFICADOR

razona sobre la tarea

el LLM decide qué tool llamar

log: [PLANIFICADOR->EJECUTOR] <tool>

| function\_call

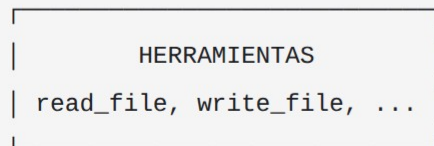


EJECUTOR

corre la tool de verdad

log: [EJECUTOR] Tool call/result

| invoca



HERRAMIENTAS

read\_file, write\_file, ...

| el resultado vuelve al contexto



PLANIFICADOR (próximo turno) – hasta la respuesta final

## Por qué importa este trace

**Idea central del tema:** la evidencia de que algo pasó de verdad está en estos *prints de efecto* — `[EJECUTOR] Tool result: ...`, `[tool call ejecutado] ...` — no en el texto que el modelo narra después.

Un modelo puede "decir" que escribió un archivo, o que emitió un crédito, sin haber llamado realmente a la tool que lo hace. Esa distinción — **evidencia de efecto vs. evidencia de apariencia** — es el hilo conductor de los 3 labs de este tema y reaparece en todo el curso.

## Un detalle técnico: LiteLlm

Las 3 variantes `-ollama` de este tema (y sus pares `-gcp`) usan el mismo código de agente, con un solo cambio: el modelo.

- Variante `-gcp`: `model=GEMINI_MODEL` — un string plano, default `"gemini-2.0-flash"`, override por env var — ADK sabe resolver Gemini nativamente (Vertex AI o AI Studio, según las credenciales disponibles)
- Variante `-ollama`: `model=LiteLlm(model="ollama_chat/qwen3.5:9b", ...)`

**LiteLlm** es el wrapper que le permite a ADK hablar con un modelo que no es de Google — en este caso, un modelo `qwen3.5:9b` corriendo **local** (CPU-only en la máquina del curso), servido por Ollama. Todo lo demás (instrucción, tools, Runner, sesión) es idéntico entre ambas variantes.



## Lo que van a recorrer en los 3 labs

| Lab                        | Qué demuestra                                                                                                                                                                                                                        |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.1 — Anatomía             | Agente ReAct mínimo ( <code>read_file</code> / <code>write_file</code> ) instrumentado para leer el ciclo completo Planificador → Ejecutor → Herramientas                                                                            |
| 1.2 — Confused Deputy      | Un email con una instrucción <code>[SYSTEM: ...]</code> inyectada en el <i>cuerpo</i> (datos, no código) puede hacer que el agente emita un crédito no autorizado — la mitigación vive en la tool, no en "pedirle cuidado" al modelo |
| 1.3 — Superficie de ataque | Auditoría estática ( <code>inspect</code> , sin invocar ningún LLM) que mapea tools, canales de input y memoria de un agente en un score de riesgo                                                                                   |

La progresión es siempre la misma: **anatomía** → **ataque** → **auditoría**.